



Lab 3 (Challenge) - Refactoring for Scale and Controlled Validation

Overview

At `c:\qatfadvez\lab3\guided`, you have been provided with a working Terraform configuration. At first glance, everything appears to function correctly - the infrastructure deploys, and there are no obvious errors. Update the subscription-id in providers.tf and prove this.

However, on closer inspection, you will notice something important.

The majority of the configuration follows strong, consistent design practices. Resources are created dynamically, inputs are structured and validated, and the overall approach is scalable and predictable.

But a recent addition — the Virtual Network and two subnets — does not follow these same standards.

These new components have been introduced in a more manual and inconsistent way. They work, but they break the established patterns of the codebase.

This creates a real-world scenario:

Someone has added infrastructure that works... but does not align with engineering standards.

Your task is to correct this.

What You Are Trying to Achieve

By the end of this lab, the entire configuration should feel consistent.

The solution should:

- scale cleanly as new inputs are added

- behave predictably when inputs change
- fail early when invalid data is provided

In short, you are moving from 'working Terraform' to 'engineered Terraform'.

Phase 1 – Refactor for Scale

In this phase, your focus is purely on design.

You are not solving Azure constraints yet. You are fixing how the Terraform is written.

Your Task

You need to refactor the VNet and subnet implementation so that it aligns with the rest of the codebase.

This means:

- replacing static subnet definitions with a dynamic approach
- ensuring resources are driven entirely from input data
- introducing subnet-to-NSG associations dynamically
- ensuring all inputs are validated before deployment

A key requirement is that your design must be stable. If input data is reordered, Terraform should not attempt to destroy and recreate resources.

A Helpful Direction (Not the Answer)

You are not starting from scratch.

Look carefully at the existing configuration.

You will find examples where:

- multiple resources are created dynamically from structured input
- input data is cleaned and validated before being used

Use these patterns as your guide when refactoring the subnet configuration.

How to Know You Are Done (Phase 1)

Your solution should behave correctly when tested:

- If you reorder the subnet input, Terraform should show no changes
- If you add a new subnet, only that subnet should be created
- If you provide invalid CIDR input, Terraform should fail during plan

If any of these behaviours are not met, your design is not yet correct.

Phase 2 – Introduce Platform Constraints

In this phase, you build on your Phase 1 solution by introducing Azure-specific rules.

Before starting, you must enable the approved CIDR validation by uncommenting the relevant variable in variables.tf

Your Task (Phase 2)

You must now enforce the following constraints:

- The VNet address space must be one of a predefined set of approved CIDRs
- Subnets must be at least /29 in size
- Subnets must always be smaller than the VNet they belong to

These rules should be enforced during terraform plan.

Hint: You'll need to extract the prefix length from a CIDR string (e.g. 10.0.1.0/24). Think about how you could split the string and work with the part after the /.

Testing Your Solution (Phase 2)

You have been provided with a set of tfvars files to test your implementation.

Use the following command: `terraform plan -var-file="<filename>.tfvars"`

You should test both valid and invalid scenarios to confirm that your validation behaves correctly.

Expected Outcomes

- The valid configuration should succeed
- Invalid configurations should fail during plan

If invalid configurations pass, your validation is incomplete.

Final Outcome

By completing this lab, you will have:

- refactored inconsistent infrastructure into a clean, scalable design
- reinforced deterministic Terraform behaviour
- implemented meaningful validation

This is a key step toward writing Terraform that is safe to use in real environments.

Copyright

© 2026 QA - Michael Coulling-Green. All rights reserved. These lab materials are provided as part of an authorised training course. Course attendees may reuse these materials for their own personal learning and reference outside of the training session. Unauthorised copying, redistribution, publication, or use of these materials to deliver training is prohibited without prior written permission from the author.

© QA 2026